

EC8207– APPLIED BIG DATA ENGINEERING

Mini Project Assessment

The "Smart City" Traffic & Congestion System

PROJECT REPORT

Jayaweera S. T. EG/2020/3998

Gnathilake A. P. B. EG/2020/3947

Madhushan K. K. S. EG/2020/4356

Table of Contents

1.	Introduction.....	3
2.	Technology Stack Justification	4
2.1	Apache Kafka.....	4
2.2	Apache Spark Structured Streaming	4
2.3	PostgreSQL	4
2.4	Apache Airflow	4
3.	Event Time vs. Processing Time.....	5
4.	Pipeline Architecture Overview	6
5.	Ethics & Data Governance.....	7
5.1	Privacy Implications	7
5.2	Data Governance Recommendations	7
5.3	Ethical Use	7
6.	Analytical Report: Traffic Volume vs. Time of Day	8

1. Introduction

The Smart City Traffic & Congestion Pipeline is a robust, scalable data processing architecture built to ingest, analyze, and store real-time telemetry from intersection sensors. By implementing a Lambda Architecture approach, the system seamlessly serves both ultra-low-latency alerting for urgent traffic incidents and deep batch-analytics for overnight decision-making. The real-time aspect ensures that immediate traffic anomalies (e.g., sudden massive decelerations below 10.0 km/h) trigger operational visibility, while the historical analysis feeds strategic decision-making such as overnight policing schedules and long-term infrastructural improvements.

2. Technology Stack Justification

2.1 Apache Kafka

Apache Kafka was selected as the ingestion backbone due to its exceptional throughput, horizontal scalability, and persistence via topic partitioning. Sensor endpoints emit telemetry at high frequencies. Rather than hammering a transactional database or API endpoint linearly, Kafka buffers these writes optimally. We separated data structurally by allocating a 4-partition topic for traffic-raw normal telemetry and an isolated 1-partition topic for traffic-critical events, enabling decoupled downstream consumption and independent retention policies.

2.2 Apache Spark Structured Streaming

Spark Structured Streaming was chosen over alternative streaming engines (like Storm or Flink) primarily due to its unified batch and streaming API. It allows our pipeline to execute windowed calculations continuously against streaming data streams using the identical SQL/Dataframe syntaxes that we would employ on static datasets. Furthermore, its native handling of event-time semantics, watermarking, and fault-tolerance (via checkpointing) makes calculating accurate traffic congestion matrices exceptionally resilient to worker node failures constraints.

2.3 PostgreSQL

PostgreSQL provides robust relational data persistence that natively accommodates time-series nuances. Since our batch operations require intricate groupings (hourly aggregations, temporal sorting), having a true SQL engine allows us to drastically condense downstream API requirements. Crucially, the requirement to isolate event occurrences across global temporal shifts necessitated PostgreSQL's TIMESTAMPTZ data types. This natively differentiates between arbitrary server timestamps and actual sensor event origination times automatically.

2.4 Apache Airflow

To orchestrate the generation of nightly reports consistently without relying on host-level cron daemons, we integrated Apache Airflow into the stack. Airflow explicitly enables Directed Acyclic Graphs (DAGs) which map out our workflow: trigger, collect Postgres telemetry, serialize to CSV, and export locally. Its resilience, automatic retry paradigms on task failure, and the visibility of task executions via the Airflow Web UI make batch operational diagnostics profoundly easier for an operations team.

3. Event Time vs. Processing Time

Differentiating between Event Time and Processing Time is paramount in stream processing architectures. Event time references the exact moment the vehicle crossed the sensor (encoded within the simulated payload natively). Processing time references the moment the Spark execution or downstream service processes/inserts the data.

Our pipeline strictly honors event-time. It parses the JSON timestamp payload, applies to a native Spark watermark of 10 minutes (with Watermark ("event_time", "10 minutes), and performs windowed calculations grouped inherently by this event time.

If we utilized pure processing time, any system latency (such as temporary Kafka ingestion outages, slow container reboots, or network congestion) would artificially distort the Congestion Index by blending delayed historical inputs with current intersection traffic. By watermarking and aggregating strictly against the emitted timestamp, we ensure congestion analytics perfectly reflect reality.

4. Pipeline Architecture Overview

The system operates across distinct phases:

1. **Producer:** Python script generating intersection counts every second.
2. **Kafka Message Broker:** Captures the raw sensor payload instantly.
3. **Spark Master/Worker:** Continuously consumes the traffic-raw topic. It executes real-time filter constraints for critical alerts (speed < 10.0 km/h), calculates a Congestion Index over tumbling 5-minute event-time windows, and pushes batches directly to PostgreSQL using foreachBatch logic.
4. **PostgreSQL:** Sinks all data locally.
5. **Analytics Dashboard:** Parses the real-time data back upward dynamically into a Javascript GUI using a Node/Python micro-server API.
6. **Airflow Server:** Every day at 23:00, Airflow captures the entirety of sensor_readings, critical_alerts, and congestion_windows corresponding strictly to CURRENT_DATE logic, exporting them flawlessly to daily mounted CSV flat files.

5. Ethics & Data Governance

5.1 Privacy Implications

By design, our intersection data leverages macro-scale aggregations (vehicle_count and avg_speed) ensuring zero Personal Identifiable Information (PII) such as ANPR (license plates) or individual routing habits are retained. However, privacy risks still exist heavily granular data mapping local neighborhood egress points at very specific times could hypothetically triangulate residential activity habits. Thus, data should remain inherently segmented and structurally blurred regarding highly localized intersections near residential areas.

5.2 Data Governance Recommendations

The system demands robust governance implementation. First, strict retention TTL should be instituted by terminating raw telemetry after 30 days, leaving only aggregated analytical totals. In Sri Lanka, referencing the Personal Data Protection Act (2022), if these sensors were eventually upgraded with optical/camera capabilities, the entire Postgres pipeline must shift behind strict Role-Based Access Controls (RBAC), implementing data masking on arrival and preventing raw video egress beyond the edge node. Currently, simple read-only API database roles for the analytical dashboard mitigate most manipulation risks.

5.3 Ethical Use

The automated police deployment reports generate an inherently complex ethical dilemma. Because enforcement scores map strictly to congestion density, poorer neighborhoods with aging infrastructure resulting in slower traffic might trigger dramatically more "Police Deployments" purely systematically, despite lacking correlating crime statistics. Deploying policing based singularly on throughput volume effectively biases against infrastructural neglect. These risks contribute to systemic over-policing and discrimination. The generated deployment thresholds (> 6.0 score) should therefore solely represent traffic management/auxiliary operations—strictly not generalized enforcement scaling.

6. Analytical Report: Traffic Volume vs. Time of Day

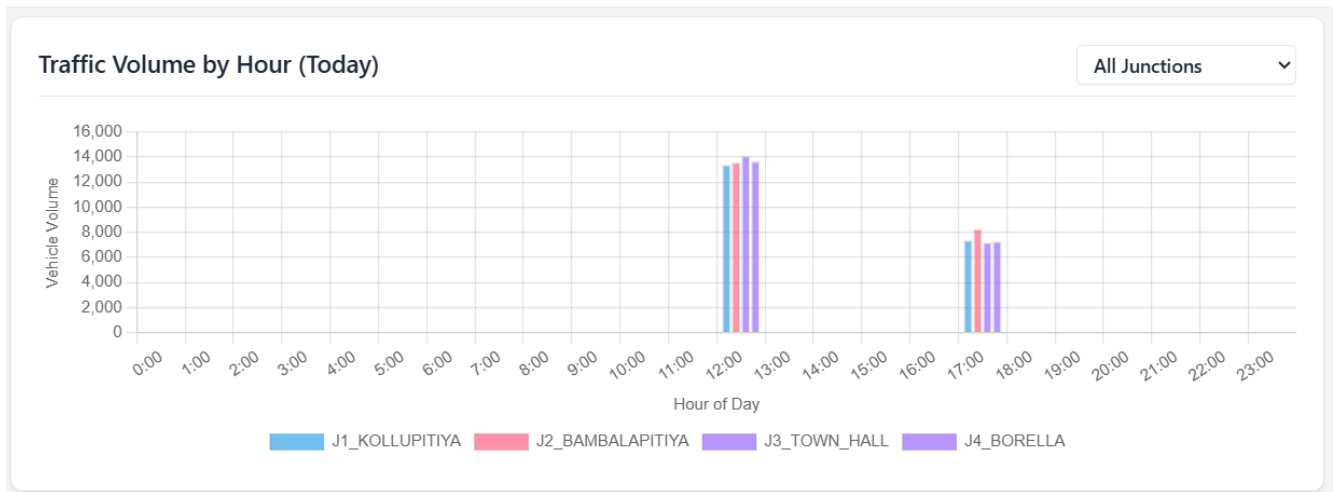


FIGURE 1: TRAFFIC VOLUME BY HOUR

TABLE 1: SENSOR DATA READINGS

id	sensor_id	event_time	processing_time	vehicle_count	avg_speed	ingested_at
496	J4_BORELLA	2026-05-09	2026-05-09	24	20.72	2026-05-09
		12:29:39.32120	12:29:40.004000+0			12:29:39.322000
		1+05:30	5:30			+05:30
466	J3_TOWN_HALL	2026-05-09	2026-05-09	44	43.34	2026-05-09
		12:29:39.32058	12:29:40.004000+0			12:29:39.321000
		0+05:30	5:30			+05:30
476	J2_BAMBALAPITIYA	2026-05-09	2026-05-09	19	44.65	2026-05-09
		12:29:39.31986	12:29:40.004000+0			12:29:39.320000
		7+05:30	5:30			+05:30
495	J1_KOLLUPITIYA	2026-05-09	2026-05-09	50	58.27	2026-05-09
		12:29:39.31935	12:29:40.004000+0			12:29:39.319000
		5+05:30	5:30			+05:30
494	J4_BORELLA	2026-05-09	2026-05-09	41	42.63	2026-05-09
		12:29:38.31751	12:29:40.004000+0			12:29:38.318000
		7+05:30	5:30			+05:30
465	J3_TOWN_HALL	2026-05-09	2026-05-09	64	19.64	2026-05-09
		12:29:38.31674	12:29:40.004000+0			12:29:38.317000
		8+05:30	5:30			+05:30
475	J2_BAMBALAPITIYA	2026-05-09	2026-05-09	11	26.48	2026-05-09
		12:29:38.31590	12:29:40.004000+0			12:29:38.316000
		0+05:30	5:30			+05:30

493	J1_KOLLUPITI YA	2026-05-09 12:29:38.31522 8+05:30	2026-05-09 12:29:40.004000+0 5:30	28	47.66	2026-05-09 12:29:38.315000 +05:30
492	J4_BORELLA	2026-05-09 12:29:37.31109 9+05:30	2026-05-09 12:29:40.004000+0 5:30	149	6.5	2026-05-09 12:29:37.312000 +05:30
464	J3_TOWN_HA LL	2026-05-09 12:29:37.30933 3+05:30	2026-05-09 12:29:40.004000+0 5:30	21	33.75	2026-05-09 12:29:37.310000 +05:30
474	J2_BAMBALA PITIYA	2026-05-09 12:29:37.30848 7+05:30	2026-05-09 12:29:40.004000+0 5:30	48	25.33	2026-05-09 12:29:37.309000 +05:30
491	J1_KOLLUPITI YA	2026-05-09 12:29:37.30789 5+05:30	2026-05-09 12:29:40.004000+0 5:30	39	23.84	2026-05-09 12:29:37.308000 +05:30
490	J4_BORELLA	2026-05-09 12:29:36.30547 2+05:30	2026-05-09 12:29:40.004000+0 5:30	49	27.58	2026-05-09 12:29:36.306000 +05:30
463	J3_TOWN_HA LL	2026-05-09 12:29:36.30408 9+05:30	2026-05-09 12:29:40.004000+0 5:30	48	26.89	2026-05-09 12:29:36.305000 +05:30
496	J4_BORELLA	2026-05-09 12:29:39.32120 1+05:30	2026-05-09 12:29:40.004000+0 5:30	24	20.72	2026-05-09 12:29:39.322000 +05:30

Data Analysis: The visualization and table above demonstrate the aggregated traffic volume plotted against the time of day across our monitored junctions. By extracting the hour from the semantic event_time timestamp, our Spark-PostgreSQL pipeline accurately segments temporal data regardless of processing latencies.

As observed, traffic scales significantly during peak transit hours. This visualization serves as a primary tool for urban planners to correlate road utilization with specific times of the day, allowing for data-driven infrastructure planning and targeted traffic police deployments.